

RABBIT HOLES

Accelerate · Manipal University Jaipur

5

PHASE

Rewriting history

Interactive rebase, and the difference between what happened and what you want to publish.



Contents

1	Two different histories	3
2	The golden rule	3
3	Interactive rebase	3
3.1	Squashing a messy branch	4
3.2	Rewording without touching content	5
3.3	Splitting one commit into two	5
4	The autosquash workflow	6
5	Cherry-pick	6
6	rebase --onto	7
7	Rewriting all of history	7
8	When a rebase goes wrong	8
9	Reference	9

1 Two different histories

The history of how you actually worked is a mess. You committed something called `wip`, then `wip 2`, then `fix typo`, then `actually fix typo`, then you realised the whole approach was wrong and started again.

The history you want a reviewer to read is a short sequence of complete, sensible changes, each of which does one thing and has a message explaining why.

Those two things are almost never the same, and git is one of very few version control systems that lets you turn the first into the second before anyone else sees it. That is what this phase is about.

WHY THIS EXISTS

This is not vanity. A clean history is a working tool. `git bisect` can find the commit that broke your build, but only if each commit builds. `git revert` can undo a feature cleanly, but only if the feature is not smeared across nine commits mixed with unrelated work. `git blame` tells you why a line exists, but only if the commit it points at has a real message. Phase 6 is entirely about tools that only work well on a tidy history. This phase is how the history gets tidy.

2 The golden rule

TRAP

Do not rewrite history that other people have.

Rewriting produces new commits with new hashes. Anyone whose branch is built on the old commits now has work attached to objects that no longer exist upstream, and sorting that out is their problem, caused by you.

In practice: rewrite freely on your own unpushed branch. Rewrite carefully on a pushed branch that only you are working on, which is what `--force-with-lease` is for. Never rewrite `main`, or any branch other people build on, unless the whole team has agreed and is standing by.

Everything below is safe on a branch that is yours alone. That covers the large majority of real use.

3 Interactive rebase

The main tool.

```
git rebase -i HEAD~5
```

This says: take the last five commits, and let me edit the plan for replaying them. Your editor

opens with a to-do list:

```
pick 3a1f2b9 Add poem parser
pick 7c4d8e1 wip
pick 2b9a0c3 wip 2
pick 8e1f4a7 fix typo
pick 5d3c2b8 Handle empty input

# Rebase 9f2e1d0..5d3c2b8 onto 9f2e1d0 (5 commands)
```

Two things surprise everyone the first time:

1. **The list is oldest first.** `git log` shows newest first. The rebase to-do shows the order commits will be replayed in, which is the opposite.
2. **You edit the list, not the commits.** You are writing a script. When you save and close, git runs it.

Change the word at the start of a line to change what happens to that commit:

Command	Effect
pick	Keep the commit as it is. The default.
reword	Keep the changes, edit the message.
edit	Stop here so you can change the commit's contents.
squash	Merge into the previous commit, and combine both messages.
fixup	Merge into the previous commit, and throw this message away.
drop	Delete the commit entirely. Deleting the line does the same.
exec	Run a shell command at this point.
break	Stop here for no reason other than that you asked.

You can also **reorder** commits by moving lines around, which git will do without complaint as long as the changes do not depend on each other.

3.1 Squashing a messy branch

The most common use by a wide margin. Turn five commits into one:

```
pick 3a1f2b9 Add poem parser
fixup 7c4d8e1 wip
fixup 2b9a0c3 wip 2
fixup 8e1f4a7 fix typo
squash 5d3c2b8 Handle empty input
```

The three `wip` messages are discarded by `fixup`. The last one is `squash`, so git will open an editor letting you write a combined message. The result is one commit called something like “Add poem parser, handling empty input”.

NOTE

`squash` and `fixup` both merge upward, into the line above. That is why the first line must stay `pick`: there is nothing above it to merge into.

3.2 Rewording without touching content

```
pick 3a1f2b9 Add poem parser
reword 7c4d8e1 asdfgh
pick 2b9a0c3 Handle empty input
```

Git replays the first commit, stops at the second, opens your editor with the old message, and continues once you save.

3.3 Splitting one commit into two

You committed two unrelated changes together and want them separated. Mark it `edit`:

```
pick 3a1f2b9 Add poem parser
edit 7c4d8e1 Add parser and fix unrelated bug
```

Git stops with that commit checked out. Undo it, keeping the changes on disk:

```
git reset HEAD~1
```

Now stage and commit the two halves separately:

```
git add parser.py
git commit -m "Add poem parser"
git add utils.py
git commit -m "Fix off-by-one in line counter"
git rebase --continue
```

TRY IT

If the two changes are in the *same file*, stage them separately with:

```
git add -p utils.py
```

Git walks you through each chunk of the diff and asks whether to stage it. Press `y` to

stage, `n` to skip, `s` to split the chunk into smaller ones, and `?` for the full list.
`git add -p` is one of the highest value commands in git and almost nobody is taught it. It is what makes small, focused commits practical instead of aspirational.

4 The autosquash workflow

Interactive rebase is powerful and it makes you stop and edit a to-do list. There is a faster path for the specific case of “I need to fix a commit I made earlier on this branch”.

Find the commit you need to amend, then:

```
git add the-fix.py
git commit --fixup 3a1f2b9
```

This makes an ordinary commit with the message `fixup! Add poem parser`. Keep working, make as many of these as you like. When you are ready to tidy up:

```
git rebase -i --autosquash HEAD~10
```

Git has already rearranged the to-do list for you: each `fixup!` commit has been moved directly beneath the commit it refers to and marked `fixup`. Usually you save the file without changing anything.

Make it the default so you never have to remember the flag:

```
git config --global rebase.autoSquash true
```

WHY THIS EXISTS

This exists because the alternative is worse. Without it you either interrupt your work to do a full interactive rebase every time you notice a small mistake, or you push a branch with six commits called “address review comments”, which tells a future reader nothing at all.

5 Cherry-pick

Take one commit from anywhere and apply it here.

```
git cherry-pick 8e1f4a7
```

Useful when a bug fix landed on a feature branch and needs to go to `main` now, without the rest of the feature.

```
git cherry-pick 8e1f4a7 2b9a0c3
git cherry-pick main~5..main~2
git cherry-pick -n 8e1f4a7
```

-n applies the change without committing, so you can adjust it first.

TRAP

Cherry-picking makes a *copy*, with a different hash. If you cherry-pick from a branch and later merge that same branch, git may present the same change twice and conflict with itself.

For one urgent fix, cherry-pick. For anything ongoing, merge properly instead. Repeated cherry-picking between long-lived branches produces duplicated history that becomes genuinely hard to reason about.

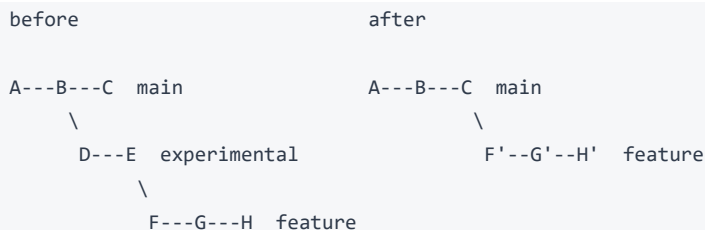
6 rebase --onto

The surgical version, and the one worth knowing when you have made a branch from the wrong place.

You branched `feature` off `experimental`, then realised it should have come off `main`. You want your three commits, without dragging `experimental` along.

```
git rebase --onto main experimental feature
```

Read it as three parts: **onto** where they should land, **from** the old base to exclude, **which** branch to move.



Commits `D` and `E` are left behind. `F`, `G` and `H` are replayed on top of `main` as new commits.

7 Rewriting all of history

Sometimes the problem is not the last few commits. Somebody committed a password three months ago, or a 200 MB video file that has been slowing every clone since.

Removing the file in a new commit is not enough. It is still in the history, still in every clone, and a secret in git history should be treated as public.

The tool is `git-filter-repo`, which is separate from git and needs installing:

```
pip install git-filter-repo
```

Remove a file from every commit that ever contained it:

```
git filter-repo --path secrets.env --invert-paths
```

Or purge a string everywhere it appears:

```
git filter-repo --replace-text expressions.txt
```

TRAP

This rewrites every commit hash in the repository. Every open pull request breaks, every clone becomes incompatible, and everyone must re-clone from scratch. It is a coordinated operation, not something to do on a Tuesday afternoon.

And it does not un-leak the secret. Anyone who cloned before you rewrote still has it, and GitHub may have cached it. **Rotate the credential first, then clean the history.** In that order, always. The rewrite is housekeeping, the rotation is the actual fix.

NOTE

You will see `git filter-branch` in older documentation. It is slow, easy to get wrong, and git itself now prints a warning recommending you use `git-filter-repo` instead. The BFG Repo-Cleaner is another good option, faster than `filter-branch` and simpler for the common cases.

8 When a rebase goes wrong

It will, and it is fine.

Command	What it does
<code>git rebase --abort</code>	Stop, put everything back the way it was
<code>git rebase --skip</code>	Discard the commit being applied, carry on
<code>git rebase --continue</code>	Carry on after resolving a conflict
<code>git rebase --edit-todo</code>	Change the plan mid-rebase
<code>git status</code>	Tells you where in the rebase you are

`--abort` works at any point and always returns you to exactly where you started. There is no state a rebase can leave you in that you cannot back out of.

If you have already finished a bad rebase:


```
git reset --hard ORIG_HEAD
```

Or find the pre-rebase position in `git reflog` and reset to it. Phase 4 covers this in full.

DEEPER

Since Git 2.26 the default rebase engine is the “merge” backend rather than the old patch based one. This is why rebase now handles renames, empty commits and directory moves far better than the reputation it earned a decade ago.

It also means `git rebase` and `git merge` share conflict resolution machinery, so `rerere` works with both. Phase 7 covers `rerere`, which remembers how you resolved a conflict and replays that resolution automatically the next time it sees the same one. On a long-lived branch that you rebase repeatedly, it is transformative.

9 Reference

Goal	Command
Edit the last commit	<code>git commit --amend</code>
Tidy the last N commits	<code>git rebase -i HEAD~N</code>
Squash a branch into one commit	<code>rebase -i</code> , then <code>fixup</code> every line but the first
Fix an earlier commit, later	<code>git commit --fixup <sha></code> then <code>rebase -i --autosquash</code>
Split a commit in two	<code>edit</code> , then <code>git reset HEAD~1</code> , then commit twice
Stage part of a file	<code>git add -p</code>
Copy one commit here	<code>git cherry-pick <sha></code>
Move a branch to a new base	<code>git rebase --onto <new> <old> <branch></code>
Remove a file from all history	<code>git filter-repo --path <f> --invert-paths</code>
Undo a finished rebase	<code>git reset --hard ORIG_HEAD</code>

CHECK YOURSELF

1. Why is the interactive rebase to-do list in the opposite order to `git log`?
2. What is the difference between `squash` and `fixup`?
3. Walk through splitting one commit into two.
4. What does `git commit --fixup` buy you over doing a rebase immediately?
5. You leaked an API key in a commit from March. What are the two steps, and which comes first?

Next: Phase 6, *Forensics*, which is about reading history rather than writing it: finding the commit that broke the build automatically, searching every version of every file for a string that no longer exists, and working out who to ask.